REST API documentation

The VUB Hub

Sofyane El Lamzi

Tariq Nassihi

Glorius Pacifique King

Tables of contents

Authentication 3
1.Registration 3
2. Login 3
3. Logout 4
4. Login Status 4
Post 5
1. Create a New Post 5
2. Get All Posts 5
3. Get Post by Post ID 6
4. Get Posts by Thread ID 6
5. Get Posts by User ID 7
6. Get Likes and Dislikes for a Post 7
7. Edit an Existing Post 7
8. Delete a Post 8
9. Fetch Posts with Geolocation (tags) 8
10. Fetch Popular Posts 9
11. Get Liked Posts of a User 9
12. Get Posts from Followed Users 9
Comments 10
1. Add a Comment to a Post 10
2. Retrieve Comments for a Post 11
3. Delete a Comment for a Post 11
4. Add a Comment to an Article 11
5. Retrieve Comments for an Article 12
6. Delete a Comment for an Article 12

Post Reactions 13
1. Like a Post 13
2. Dislike a Post 13
3. Remove a Reaction on a Post 14
4. Get Total Likes and Dislikes for a Post 14
5. Get the Current User's Reaction to a Post 15
6. React to a Comment 15
7. Get Reaction Count for a Comment 15
8. Get the Current User's Reaction to a Comment 16
9. Remove a Reaction to a Comment 16
News Articles 17
1. Fetch News Articles 17
Search 17
1.Search Users, Threads, and Posts 17
Threads 18
1. Create a New Thread 18
2. Fetch All Threads 18
3. Fetch a Thread by ID 19
User 19
1. Get All Users 19
2. Get Specific User or Current User 19
3. Follow a User 20
4. Unfollow a User 21
5. Update User Profile Image 21
6. Update User Name 22

- **Base URL :** http://localhost:8000/
- **No headers are required for any requests**
- **All request bodies and responses are in JSON format.**

Authentication

1.Registration:

Registers a new user.

- **Request**
  - http request: “POST base_url/auth/register”
  - Body : {"name": "string", "email": "string", "password": "string"}
- **Response**
  - **Success (201):** {"message": "User registered successfully"}
  - **Error (400) :**
    When validation errors are present:

    {"errors": [{"msg": "string", "param": "string", "location": "string"}]}

    When the user already exists:

    {"error": "User already exists"}

  - **Error (500):** server error **:** {"error": "An error occurred during registration"}
- **Validation:**
  - name: Required, must be a string between 3 and 15 digits
  - email: Required, must be a valid email address.
  - password: Required, must be at least 6 characters long.

2. Login:

Logs in an existing user.

- **Request**
  - http request: “POST base_url/auth/login”
  - Body: { "email": "string", "password": "string"}

- **Response**
  - **Success (200):** {"message": "User logged in successfully", "token": true}
  - **Error (500):** server error **:** { "error": "An error occurred during login"}
  - **Error (400):** When validation errors are present:

    {"errors": [{"msg": "string", "param": "string", "location": "string"}]}

- **Validation:**
    - email: Required, must be a valid email address.
    - password: Required, must be a string.

3. Logout:

Logs out the current user.

- **Request :**
    - http request: “POST base_url/auth/logout”
    - Body: none

- **Response :**
    - **Success (200):**{"message": "Logged out successfully"}
    - **Error (401)** user is not logged-in: {"error": "Unauthorized: You are not logged in"}
    - **Error (500)** server error **:**{ "error": "An error occurred during logout"}

4. Login Status:

Check if the user is logged in.

- **Request :**
    - http request: “GET base_url/auth/login/status”
    - Body: none
- **Response :**
    - **Success (200):** {"code": 200, "message": "User is authenticated", "user":{"id":"number", "name": "string", "email": "string", “image_profile_url”: “string”}}
    - **Error (401):** user is not logged-in: {"message": "User is not authenticated"}

Post

1. Create a New Post

- **Request:**
    - http request: “POST base_url/posts/”
    - Body: {"title": "string", "content": "string", "image": "string", "tags": "string", "threadId": "number"}

- **Response:**
    - **Success (201):**

      { "message": "Post created successfully", "postId": "number", "title": "string", "content": "string", "image": "string", "tags": "string", "authorId": "number", "threadId": "number"}
    - **Error (400**) validation error:

      { "errors": [{"msg": "string", "param": "string", "location": "string"}]}
    - **Error (401)** user is not logged-in:

      {"error": "Unauthorized: Please log in first"}
    - **Error (404)** when thread doesn’t exist:

      {"error": "Thread not found"}
    - **Error (500)** server error **:**

      {"error": "An error occurred while creating the post"}

- **Validation**
    - title: required,must be a non-empty string, with a length between 5 and 50 characters
    - content: required,must be a non-empty string, with a length between 5 and 500 characters
    - tags: option,must be a valid string (if provided)
    - image: option, must be a valid URL (if provided)
    - threadId: required, intiger.

2. Get All Posts

- **Request:**
    - http request: “GET base_url/posts/”
    - Body: none

- **Response:**
    - **Success (200)**:

      [{ "id": "number", "content": "string", "created_at": "string", "title": "string", "image_url": "string", "tags": "string", "reading_time": "number", "username": "string", "profile_img": "string" }]
    - **Error (404)** when there is no posts:

      {"error": "No posts found"}
    - **Error (500)** server error **:**

      {"error": "An error occurred while getting the posts"}

3. Get Post by Post ID

- **Request:**
    - http request: “GET base_url/posts/:postId”
    - Body: none
    - Parameters: id of the post
- **Response:**
    - **Success (200):**

      { "id": "number", "title": "string", "content": "string", "image_url": "string", "tags": "string", "author_name": "string", "author_image": "string","authorId" : int, "created_at" : string, "threadId" : Int}
    - **Error (404)** post doesn’t exist: {"error": "Post not found"}
    - **Error (500)** server error**:** {"error": "An error occurred while getting the post"}

4. Get Posts by Thread ID

- **Request:**
    - http request: “GET base_url/posts/:threadId”
    - Body: none
    - Parameters: id of the thread
- **Response:**
    - **Success (200)**:

      { "id": "number", "title": "string", "content": "string", "image_url": "string", "tags": "string", "author_name": "string", "author_image": "string","authorId" : int, "created_at" : string, "threadId" : Int}
    - **Error (500)** server error**:** {"error": "An error occurred while getting the posts"

5. Get Posts by User ID

- **Request:**
  - http request: “GET base_url/posts/:userId”
  - Body: none
  - Parameters: id of the user
- **Response:**
  - **Success (200):**

    { "id": "number", "title": "string", "content": "string", "image_url": "string", "tags": "string", "author_name": "string", "author_image": "string","authorId" : int, "created_at" : string, "threadId" : Int}
  - **Error (500)** server error**:** {"error": "An error occurred while getting the posts"}

6. Get Likes and Dislikes for a Post

- **Request:**
  - http request: “GET base_url/posts/reaction/:postId
  - Body: none
  - Parameters: id of the post
- **Response:**
  - **Success (200):**

    { "message": "Reactions retrieved successfully", "reactions": [

    {"reaction": "string", "user_name": "string"}]}
  - **Error (404):** when no reactions: {"error": "No reactions found for this post"}
  - **Error (500)** server error**:** {"error": "An error occurred while retrieving the reactions"}

7. Edit an Existing Post

- **Request:**
  - http request: “PUT base_url/posts/ postId
  - Body: { "title": "string", "content": "string", "image_url": "string", "tags": "string"}
  - Parameters: id of the post
- **Response:**
  - **Success (200**): {"message": "Post updated successfully"}
  - **Error (400)** validation error: {"errors": [{"msg": "string", "param": "string", "location": "string"}]}
  - **Error (401)** user is not logged-in: {"error": "Unauthorized: Please log in first"}

- **Error (403)** user try to edit another user post : {"error": "You can only edit your own posts"}
- **Error (404)** when post doesn't exist : {"error": "Post not found"}
- **Error (500)** server error: {"error": "An error occurred while updating the post"}

- **Validations:**
  - title: required,must be a non-empty string, with a length between 5 and 50 characters
  - content: required,must be a non-empty string, with a length between 5 and 500 characters
  - tags: option,must be a valid string (if provided)
  - image: option, must be a valid URL (if provided)
  - threadId: required, intiger.

8. Delete a Post

- **Request:**
  - http request: "DELETE base_url/posts/ postId
  - Parameters: id of the post
  - Body: none
- **Response:**
  - **Success (200):** {"message": "Post deleted successfully"}
  - **Error (401)** user is not logged-in: {"error": "Unauthorized: Please log in first"}
  - **Error (403)** user try to delete another user post {"error": "You can only delete your own posts"}
  - **Error (404)** when post doesn't exist : {"error": "Post not found"}
  - **Error (500)** server error: {"error": "An error occurred while deleting the post"}

9. Fetch Posts with Geolocation (tags)

- **Request:**
  - http request: "GET base_url/posts /post-with-locations
  - Body: none
- **Response:**
  - **Success (200):**

    { "id": "number", "content": "string", "created_at": "string", "title": "string", "image_url": "string", "tags": "string | null", "reading_time": "number", "username": "string", "user_id": "number", "profile_img": "string", "location": { "lat": "number", "lng": "number" } | null }

    Error (500) server error: {"error": "Error processing blogs with locations"}

10. Fetch Popular Posts

- **Request:**
  - http request: “GET base_url/posts /popular-posts
  - Body: none
- **Response:**
  - **Success (200):**

    {"id": "number", "title": "string", "content": "string", "created_at": "string", "image_url": "string", "profile_img": "string", "tags": "string ", "username": "string", "user_id": "number","like_count": "number"}
  - **Error (500)** server error:

    {"error": "Error fetching popular posts"}

11. Get Liked Posts of a User

- **Request:**
  - http request: “GET base_url/posts / liked-posts/:userId
  - Parameters: id of the user
  - Body: none
- **Response:**
  - **Success (200):**

    [{ "id": "number", "content": "string", "created_at": "string", "title": "string", "image_url": "string", "tags": "string", "reading_time": "number", "username": "string", "profile_img": "string" }]
  - **Error (401)** user is not logged-in:

    {"error": "Unauthorized: Please log in first"}
  - **Error (500)** server error :

    {"error": "An error occurred while fetching liked posts"}

12. Get Posts from Followed Users

- **Request:**
  - http request: “GET base_url/posts /followed-posts
  - body: none

- **Response:**
    - **Success (200):**

      { "title": "string", "id": "number", "content": "string", "username": "string", "user_id": "number", "created_at": "string", "image_url": "string"," profile_img": "string"}
    - **Error (401)** user is not logged-in:

      {"error": "Unauthorized: Please log in first"}
    - **Error (500)** server error

      {"error": "Failed to fetch followed posts"}

Comments

1. Add a Comment to a Post

- **Request:**
    - http request: “POST base_url/comments/ post/:postId
    - Body: { "content": "string"}
    - Parameters: id of the post
- **Response:**
    - **Success (201):**

      {"message": "Comment added successfully", "commentId": "number", "content": "string"}
    - **Error (400)** validation error :

      {"errors": [{"msg": "string", "param": "string", "location": "string"}]}
    - **Error (401)** user is not logged-in:

      {"error": "Unauthorized: Please log in first"}
    - **Error (500)** server error :

      {"error": "An error occurred while adding the comment"}
- **Validations:**
    - content: required, must be a non-empty string, with a length between 3 and 500 characters.

2. Retrieve Comments for a Post

- **Request:**
  - http request: “GET base_url/comments/:postId
  - body : none
- **Response:**
  - **Success (200):**

    [{"id": "number", "content": "string", "created_at": "string", "user_name": "string", "user_image": "string", "user_id": "number"}]
  - **Error (500)** server error:

    {"error": "An error occurred while retrieving the comments"}

3. Delete a Comment for a Post

- **Request:**
  - http request: “DELETE base_url/comments/post/:postId/:commentId
  - parameters : id of the post and comment
  - body: none
- **Response:**
  - **Success (200):**

    {"message": "Comment deleted successfully"}
  - **Error (401)** user is not logged-in:

    {"error": "Unauthorized: Please log in first"}
  - **Error (403)** user try to delete another user comment:

    {"error": "You can only delete your own comments"}
  - **Error (404)** when comment doesn’t exist:

    {"error": "Comment not found"}
  - **Error (500)** serve error :

    {"error": "An error occurred while deleting the comment"}

4. Add a Comment to an Article

- **Request:**
  - http request: “POST base_url/comments/article/:articleId
  - Body: {"user": "number", "content": "string"}
  - Parameters: id of the article

- **Response:**
    - **Success (201):**

      {"message": "Comment added successfully", "content": "string"}
    - **Error (400)** validation error :

      {"errors": [{"msg": "string", "param": "string", "location": "string"}]}
    - **Error (404)** when article doesn't exist:

      {"error": "Artikel not found"}
    - **Error (500)** server error :

      {"error": "Fout bij het toevoegen van de comment"}
- Validations:
    - content: required, must be a non-empty string, with a length between 3 and 500 characters.

5. Retrieve Comments for an Article

Request:

- http request: "GET base_url/comments/article/:articleId
- parameters: id of the article
- body: none

- **Response:**
    - **Success (200):**

      [{"id": "number", "content": "string", "date": "string", "user_name": "string", "image_profile": "string"}]
    - **Error (404)** when there are no comments:

      {"error": "No comments found for this article"}
    - **Error (500)** server error :

      {"error": "An error occurred while retrieving the comments"}

6. Delete a Comment for an Article

- **Request:**
    - http request: "DELETE base_url/comments/article/:articleId/:commentId
    - parameters: id of the article and the comment
    - body: none

- **Response:**
    - Success (200):

      {"message": "Comment deleted successfully"}
    - Error (404) when the comment doesn't exist:

      {"error": "Comment not found"}
    - Error (500) server error :

      {"error": "An error occurred while deleting the comment"}
- **Response :**
    - Success (200): {"message": "Logged out successfully"}
    - Error (401): { "error": "Unauthorized: You are not logged in"}
    - Error (500):{ "error": "An error occurred during logout"}

Post Reactions

1. Like a Post

- **Request:**
    - http request: "POST base_url/react/ like/:postId
    - parameters: id of the liked post
    - body: none
- **Response:**
    - **Success (200):** { Reaction updated to 'like' successfully}
    - **Success (201):** {Post successfully liked.}
    - **Error(401)** user is not logged-in:

      { error: 'Unauthorized: Please log in first' }
    - **Error (500)** Server error : { error: 'An error occurred while liking the post' }
    -

2. Dislike a Post

- **Request:**
    - http request: "POST base_url/react/ dislike/:postId
    - parameters: id of the disliked post
    - body: none

- **Response:**
    - **Success (200):** { Reaction successfully updated to 'dislike'}
    - **Success(201):** {Post successfully disliked}
    - **Error(401)** user is not logged-in:

      { error: 'Unauthorized: Please log in first' }
    - **Error (500)** Server error : { error: 'An error occurred while disliking the post' }

3. Remove a Reaction on a Post

- **Request :**
    - http request: “DELETE base_url/react/:postId
    - parameters: id of the post
    - body: none
- **Response:**
    - **Success (200):** {Reaction successfully removed}
    - **Error(401):** user is not logged-in:

      { "error": "Unauthorized: Please log in first" }
    - **Error(404)** : {No reaction found to remove}
    - **Error (500)** Server error: { error: 'An error occurred while removing the reaction' }

4. Get Total Likes and Dislikes for a Post

- **Request**
    - http request: “GET base_url/react/ count/:postId
    - parameters: id of the post
    - body: none
- **Response:**
    - **Success (200):** Provides a summary of likes and dislikes.
    - **Error (500)** Server error: { error: 'An error occurred while fetching the reactions count' }
    -

5. Get the Current User's Reaction to a Post

- **Request**
  - http request: “GET base_url/react/ reaction/:postId
  - parameters: id of the post
  - body: none
- **Response:**
  - **Success (200):** Returns the reaction ('like' or 'dislike').
  - **Error(404):** {No reaction found}
  - **Error(401):** user is not logged-in:

    { "error": "Unauthorized: Please log in first" }
  - **Error (500)** Server error : { error: 'An error occurred while fetching the reaction' }

6. React to a Comment

- **Request**
  - http request: “POST base_url/react/comments/react/:commentId
  - parameters: id of the comment
  - body: none
- **Response:**
  - **Success (200):** {Reaction successfully updated}
  - **Success(201):** {Reaction successfully added}
  - **Error(401):** user is not logged-in:

    { "error": "Unauthorized: Please log in first" }
  - **Error (500)** Server error : { error: 'An error occurred while reacting to the comment' }

7. Get Reaction Count for a Comment

- **Request**
  - http request: “GET base_url/react/ comments/reaction-count/:commentId
  - parameters: id of the comment
  - body: none

- **Response**
    - **Success (200):** Provides a summary of likes and dislikes.
    - **Error(401):** user is not logged-in:

      { "error": "Unauthorized: Please log in first" }
    - **Error (500)** Server error : { error: 'An error occurred while fetching reactions }

8. Get the Current User's Reaction to a Comment

- **Request**
    - http request: “get base_url/react/ comments/reaction/:commentId
    - parameters: id of the comment
    - body: none

- **Response:**
    - **Success (200):** Returns the reaction ('like' or 'dislike').
    - **Error(404)** : {No reaction found}
    - **Error(401):** user is not logged-in:

      { "error": "Unauthorized: Please log in first" }
    - **Error (500)** Server error : { error: 'An error occurred while fetching the reaction' }

9. Remove a Reaction to a Comment

- **Request**
    - http request: “DELETE base_url/react/comments/reaction/:commentId
    - parameters: id of the removed comment
    - body: none
- **Response:**
    - **Success (200):** Reaction successfully removed.
    - **Error(401):** user is not logged-in:

      { "error": "Unauthorized: Please log in first" }
    - **Error(404):** No reaction found to remove.
    - **Error (500):** Server error : { error: 'An error occurred while removing the reaction' }

News Articles

1. Fetch News Articles

Fetch news articles from external api (news api) (related to "VUB", "students", and general news in Belgium) and save them to the database. Limits the total articles to 50.

- **Request**
  - http request: “GET base_url/news
  - body: none
- **Response:**
  - **Success (200):** Array of articles
  - **Error (500)** Server error: 'Error getting news'

Search

1.Search Users, Threads, and Posts

- **Request**
  - http request: “GET base_url/search/
  - body: none
- **Response:**
  - **Success (200):**

    {"message": "Search results retrieved successfully", "results":

    {"users": [{ "id": Int, "name": "String" , "image_profile_url": " String"" }],

    "threads": [{ "id": Int, "title": " String"" }],

    "posts": [{ "id": Int, "title": " String"" }]}
  - **Error (400)** when no query is given

    { "error": "Query parameter is required" }
  - **Error (500**) server error

    { "error": "An error occurred while retrieving search results" }

Threads

1. Create a New Thread

- **Request**
  - http request: “POST base_url/threads/
  - Body: { "title": "string"}

- **Response:**
  - **Success (200):**

    { "message": "Thread created successfully", "threadId": int, "title": "string"}
  - **Error(400)** Validation errors :

    {"errors": [{"msg": "string", "param": "string", "location": "string"}]}
  - **Error(401)** user is not logged-in:

    { "error": "Unauthorized: Please log in first" }
  - **Error(409)** Conflict, thread title already exists:

    { "error": "A thread with this title already exists" }
  - **Error (500)** Server error : { error: 'An error occurred while creating the thread' }
- **Validations**:
  - title: required, must be a non-empty string, with a length between 5 and 50 characters.

2. Fetch All Threads

- **Request**
  - http request: “GET base_url/threads/
  - body: none
- **Response:**
  - **Success (200)** List of threads :

    [{ "id": int, "title": "string", "user_id": int }]
  - **Error (500)** Server error:

    {"error": "An error occurred while fetching threads" }

3. Fetch a Thread by ID

- **Request**
  - http request: “GET base_url/threads/:threadId
  - body: none
  - parameters: id of the thread
- **Response:**
  - **Success (200)** Thread details :

    [{ "id": int, "title": "string", "user_id": int }]
  - **Error(404)** Thread not found :

    { "error": "No thread found" }
  - **Error (500)** Server error :

    { "error": "An error occurred while getting the thread"

User

1. Get All Users

- **Request**
  - http request: “GET base_url/user/
  - body: none
- **Response:**
  - **Success (200):** List of users.

    [{ "id": int, "name": "string", "email": "string", "image_profile_url": "string”]
  - **Error(404)** No users found: { "error": "No users found" }
  - **Error (500)** Server error. : { error: 'An error occurred while fetching users' }

2. Get Specific User or Current User

- **Request**
  - http request: “GET base_url/user/:userId
  - body: none
  - parameters: id of the user

- **Response:**
    - **Success (200):** User details.

      { "id": int, "name": "string", "email": "string", "image_profile_url": "string", "followers": [user-id's], "following": [user-id's], "likedPosts": [post-id's],"dislikedPosts": [post-id's]}
    - **Error(401)** user is not logged-in:

      {"error": "Unauthorized: Please log in first" }
    - **Error(404)** User not found :

      { "error": "User not found" }
    - **Error (500)** Server error. :

      { error: 'An error occurred while getting user data' }

3. Follow a User

- **Request**
    - http request: "POST base_url/user/ follow/:username
    - body: none
    - parameters: name of the followed user
- **Response:**
    - **Success (201)**:.

      {"message": "User followed successfully" }
    - **Error(400)** already follow or following self:

      { "error": "You cannot follow yourself" }

      { "error": "You are already following this user" }
    - **404:** User not found.

      { "error": "User not found" }
    - **Error (500)** Server error :

      { error: 'An error occurred while following the user' }

4. Unfollow a User

- **Request**
  - http request: “DELETE base_url/user/unfollow/:username
  - body: none
  - parameters: name of the unfollowed user
- **Response:**
  - **Success (200):** { "message": "User unfollowed successfully" }
  - **Error(404)** User not found :

    { "error": "User not found" }
  - **Error (500)** Server error :

    { error: 'An error occurred while unfollowing the user' }

5. Update User Profile Image

- **Request**
  - http request: “PUT base_url/user/update-profile-img
  - Body: {"image_profile_url": "string"}
- **Response:**
  - **Success (200):**

    { "message": "Profile image updated successfully" }
  - **Error(400)** Validation errors.

    {"errors": [{"msg": "string", "param": "string", "location": "string"}]}
  - **Error(401)** user is not logged-in:

    { "error": "Unauthorized: Please log in first" }
  - **Error (500)** Server error : ({ error: 'An error occurred while updating the profile image' }
- Validations:
  - image_profile_url: required, must be a valid URL.

6. Update User Name

- **Request**
  - http request: “PUT base_url/user/ update-name
  - Body: { “name": "string"}

- **Response:**
  - **Success (200):**

    { "message": "Name updated successfully" }
  - **Error(400)** Validation errors.

    {"errors": [{"msg": "string", "param": "string", "location": "string"}]}
  - **Error(401)** user is not logged-in:

    { "error": "Unauthorized: Please log in first" }
  - Error (500) Server error :

    { error: 'An error occurred while updating the name' }

- Validations;
  - name: required, must be a non-empty string, with a length between 3 and 15 characters.